

Q2. Kruskal vs Boruvka Algorithm (MST Efficiency)

Aim:

To construct the Minimum Spanning Tree (MST) using Kruskal's and Borůvka's Algorithms and compare their efficiency.

Given Graph:

Edges:

(A-B:1)

(B-C:4)

(C-D:2)

(D-A:3)

(B-D:5)

Kruskal's Algorithm:

Step 1: Sort edges

Edge	Weight
A-B	1
C-D	2
D-A	3
B-C	4
B-D	5

Step 2: Select edges

1. A-B (1) ✓
2. C-D (2) ✓
3. D-A (3) ✓

MST obtained.

MST:

Edges:

- A-B (1)
- C-D (2)
- D-A (3)

Total Cost = $1 + 2 + 3 = 6$

Borůvka's Algorithm:

Iteration 1

Each vertex selects its cheapest edge:

Vertex	Cheapest Edge
A	A-B (1)
B	A-B (1)
C	C-D (2)
D	C-D (2)

Selected:

- A-B (1)
- C-D (2)

Components:

- {A,B}
- {C,D}

Iteration 2

Cheapest edge connecting components:

- D-A (3)

Selected:

- D-A (3)

Single component formed.

MST

Edges:

- A-B (1)
- C-D (2)
- D-A (3)

Total Cost = **6**

Comparison:

Feature	Kruskal	Borůvka
Approach	Global edge sorting	Cheapest edge per component
Iterations	3 selections	2 iterations
Data Structure	Union-Find	Component tracking
Time Complexity	$O(E \log E)$	$O(E \log V)$
MST Cost	6	6

Sample Output:

Minimum Spanning Tree:

A-B (1)

C-D (2)

D-A (3)

Total Weight = 6

Time and Space Complexity:**Kruskal**

- Time Complexity: $O(E \log E)$
- Space Complexity: $O(V)$

Borůvka

- Time Complexity: $O(E \log V)$
- Space Complexity: $O(V)$

Result:

Both Kruskal's and Borůvka's Algorithms produce the same MST with total weight **6**. Kruskal relies on globally sorted edges, while Borůvka repeatedly selects the cheapest edge from each component, often reducing the number of iterations for large graphs.